

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА



Институт радиоэлектроники и информационных технологий

«Создание Telegram Bot»

ОТЧЕТ

по лабораторной работе № 4

по дисциплине

Программирование на Python

РУКОВОДИТЕЛЬ:

(подпись)

Зуев А. Б.
(Фамилия, И., О.)

СТУДЕНТ:

(подпись)

Рябков Н.А.
(Фамилия, И., О.)

24-ИСТ-2
(шифр группы)

Работа защищена «__» _____

С оценкой _____

Нижний Новгород

2026

Содержание

ОТЧЕТ	1
Цели и задачи.....	3
Описание проделанной работы	4
Вывод.....	7
Листинг программы	8

					ЛР4 – НГТУ – 24-ИСТ-2						
Изм	Лист	№ Докум.	Подпись	Дата							
Разраб.	Акифьев И.И.				Лабораторная работа №4			Лит.	Лист	Листов	
Проверил	Зуев А.Б.									2	8
								Кафедра ЭСВМ гр. 24-ИСТ-2			
Н. контр.											
Утв.	Зуев А.Б.										

Цели и задачи

Цель работы: Составить программу на Python, которая реализует Telegram Bot, определяющий государственные праздники России в любой день года.

Описание проделанной работы

Для работы с Telegram Bot`ом необходимо создать его при помощи стороннего Telegram Bot`а - “Bot Father” (см. Рис. 1). Благодаря нему мы сможем получить токен для дальнейшей работы с нашим ботом.

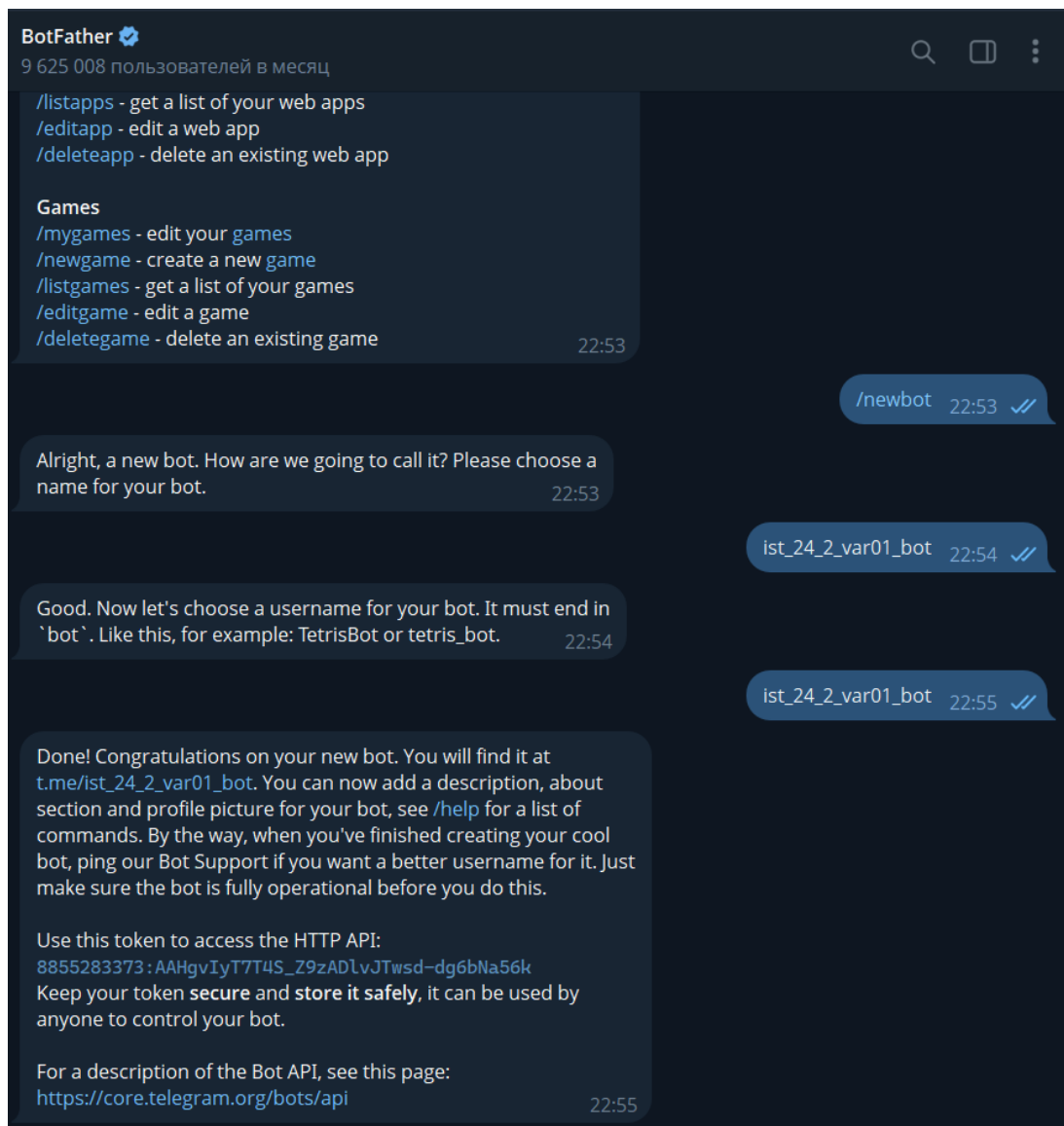


Рисунок 1. Создание Telegram Bot.

Токен был сохранён в файле .env для обеспечения безопасности и исключения его попадания в публичный репозиторий.

Для работы с ботом, а именно разработки его функционала при помощи языка программирования Python, необходимо импортировать

					ЛР4 – НГТУ – 24-ИСТ-2	Лист
Изм.	Лист	№ докум.	Подпись	Дата		4

библиотеку telegram (python-telegram-bot). Для инициализации бота в нашем Python проекте необходимо задать переменную BOT_TOKEN, которая будет принимать токен из "Bot Father". Для создания объекта бота используем Application.builder().token(BOT_TOKEN).build() (см. Рис. 2).

```
# Создаём приложение
application = Application.builder().token(BOT_TOKEN).build()
```

Рисунок 2. Инициализация Telegram Bot`а.

Для взаимодействия с ботом используются обработчики команд. В обработчике команды /start вызывается функция start, которая при помощи параметра update.message.reply_text отправляет приветственное сообщение с именем пользователя и показывает кнопки (см. Рис. 3).

```
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE): 1 usage
    """Приветствие по имени"""
    user_name = update.effective_user.first_name
    keyboard = [
        [InlineKeyboardButton(text="Праздник сегодня", callback_data="today")],
        [InlineKeyboardButton(text="Проверить другую дату", callback_data="check")],
        [InlineKeyboardButton(text="Помощь", callback_data="help")]
    ]
    reply_markup = InlineKeyboardMarkup(keyboard)
    await update.message.reply_text(
        text=f"Привет, {user_name}!\n\n"
        f"Я бот для поиска праздников в России.\n"
        f"Нажми /help для команд.",
        reply_markup=reply_markup
    )
```

Рисунок 3. Обработчик команды “/start”.

Для получения данных о праздниках используется публичный API date.nager.at. Реализуем функцию get_holiday, которая делает запрос к API, получает список праздников за указанный год и отбирает те, которые приходятся на заданную дату (см. Рис. 4). Для работы с датами используется

библиотека datetime.

```
def get_holiday(date_str: str = None) -> str: 3+ usages
    """Получает информацию о праздниках"""
    if date_str is None:
        date_str = datetime.now().strftime('%Y-%m-%d')

    try:
        year = date_str[:4]
        url = f"https://date.nager.at/api/v3/PublicHolidays/{year}/RU"
        response = requests.get(url, timeout=15)
        response.raise_for_status()
        holidays = response.json()

        holidays_on_date = [h for h in holidays if h['date'] == date_str]

        if holidays_on_date:
            holiday_names = [h['localName'] for h in holidays_on_date]
            return f"<b>{date_str}</b>\n\nПраздник: {' ', ' '.join(holiday_names)}"
        else:
            return f"<b>{date_str}</b>\n\nНа эту дату государственных праздников в России нет."

    except requests.exceptions.RequestException:
        return "Не удалось получить данные о праздниках. Проверьте интернет."
```

Рисунок 4. Функция получения праздников из API.

Для обработки ввода пользователя реализован обработчик текстовых сообщений. Бот ожидает дату в формате ГГГГ-ММ-ДД. Если формат верный, вызывается get_holiday и возвращается результат. Если нет – выводится сообщение об ошибке (см. Рис. 5).

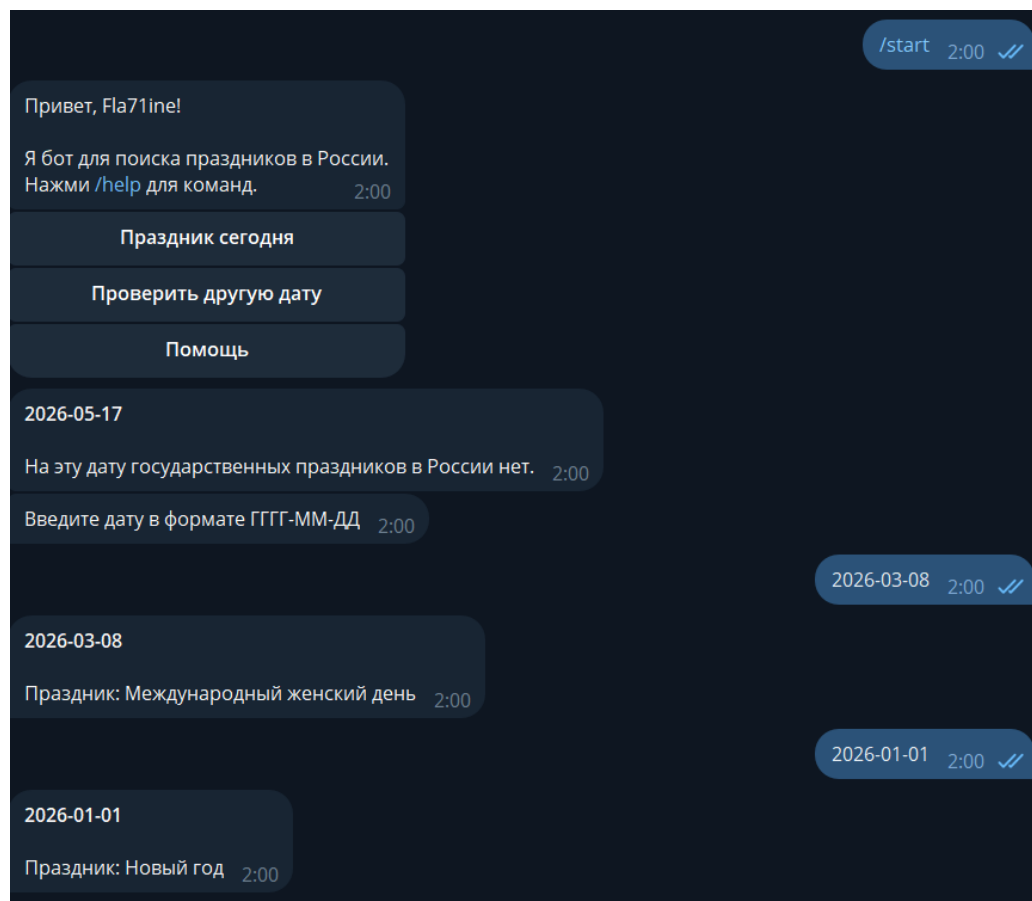


Рисунок 5. Пример входных и выходных данных в Telegram Bot.

Для запуска и непрерывной работы бота используется метод `application.run_polling()` (см. Рис. 6).

```
# Запускаем бота
print("Бот готов к работе! Напишите @ваш_username_бота в Telegram")
application.run_polling()
```

Рисунок 6. Запуск бота.

Вывод

В результате выполнения лабораторной работы мною были решены следующие задачи: разработка собственного Telegram Bot'a, который определяет государственные праздники России в любой день года. Бот приветствует пользователя по имени, использует кнопки для минимизации ввода, содержит команду `/help`, а данные берутся с внешнего web-API.

Листинг программы

```
import os
import logging
import requests
from datetime import datetime
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import Application, CommandHandler, CallbackQueryHandler,
MessageHandler, filters, ContextTypes
from dotenv import load_dotenv

load_dotenv()
BOT_TOKEN = os.getenv('BOT_TOKEN')

if not BOT_TOKEN:
    raise ValueError("BOT_TOKEN не найден!")

logging.basicConfig(level=logging.INFO)

def get_holiday(date_str: str = None) -> str:
    """Получает информацию о праздниках"""
    if date_str is None:
        date_str = datetime.now().strftime('%Y-%m-%d')

    try:
        year = date_str[:4]
        url = f"https://date.nager.at/api/v3/PublicHolidays/{year}/RU"
        response = requests.get(url, timeout=15)
        response.raise_for_status()
        holidays = response.json()

        holidays_on_date = [h for h in holidays if h['date'] == date_str]

        if holidays_on_date:
            holiday_names = [h['localName'] for h in holidays_on_date]
            return f"<b>{date_str}</b>\n\nПраздник: {', '.join(holiday_names)}"
        else:
```



```
        return f"<b>{date_str}</b>\n\nНа эту дату государственных  
праздников в России нет."
```

```
    except requests.exceptions.RequestException:  
        return "Не удалось получить данные о праздниках. Проверьте интернет."
```

```
def is_valid_date(date_string: str) -> bool:  
    try:  
        datetime.strptime(date_string, '%Y-%m-%d')  
        return True  
    except ValueError:  
        return False
```

```
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):  
    """Приветствие по имени"""  
    user_name = update.effective_user.first_name  
    keyboard = [  
        [InlineKeyboardButton("Праздник сегодня", callback_data="today")],  
        [InlineKeyboardButton("Проверить другую дату",  
callback_data="check")],  
        [InlineKeyboardButton("Помощь", callback_data="help")]  
    ]  
    reply_markup = InlineKeyboardMarkup(keyboard)  
    await update.message.reply_text(  
        f"Привет, {user_name}!\n\n"  
        f"Я бот для поиска праздников в России.\n"  
        f"Нажми /help для команд.",  
        reply_markup=reply_markup  
    )
```

```
async def help_command(update: Update, context: ContextTypes.DEFAULT_TYPE):  
    """Справка"""  
    await update.message.reply_text(  
        "Команды:\n"  
        "/start - Приветствие\n"  
        "/help - Помощь\n"
```

```
"/today - Праздники сегодня\n\n"  
"Формат даты: ГГГГ-ММ-ДД\n"  
"Пример: 2025-01-01"  
)
```

```
async def today(update: Update, context: ContextTypes.DEFAULT_TYPE):  
    """Праздники сегодня"""  
    holiday_info = get_holiday()  
    await update.message.reply_text(holiday_info, parse_mode='HTML')
```

```
async def check(update: Update, context: ContextTypes.DEFAULT_TYPE):  
    """Запрос даты"""  
    await update.message.reply_text(  
        "Введите дату в формате ГГГГ-ММ-ДД:\nПример: 2025-01-01"  
    )
```

```
async def button_callback(update: Update, context: ContextTypes.DEFAULT_TYPE):  
    """Обработка кнопок"""  
    query = update.callback_query  
    await query.answer()  
  
    if query.data == "today":  
        holiday_info = get_holiday()  
        await query.message.reply_text(holiday_info, parse_mode='HTML')  
    elif query.data == "check":  
        await query.message.reply_text("Введите дату в формате ГГГГ-ММ-ДД")  
    elif query.data == "help":  
        await query.message.reply_text(  
            "Просто отправь дату в формате ГГГГ-ММ-ДД\n"  
            "или используй команды: /today, /check"  
        )
```

```
async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):  
    """Обработка текстовых сообщений"""
```

```

text = update.message.text.strip()

if is_valid_date(text):
    holiday_info = get_holiday(text)
    await update.message.reply_text(holiday_info, parse_mode='HTML')
else:
    await update.message.reply_text(
        "Неверный формат!\n"
        "Используйте: ГГГГ-ММ-ДД\n"
        "Пример: 2025-01-01\n\n"
        "Или нажмите /today"
    )

def main():
    """Запуск бота"""
    print("Бот запускается...")
    print("Библиотека: python-telegram-bot 20.x")
    print("Вариант №1: Праздники в любой день года")

    # Создаём приложение
    application = Application.builder().token(BOT_TOKEN).build()

    # Регистрируем обработчики
    application.add_handler(CommandHandler("start", start))
    application.add_handler(CommandHandler("help", help_command))
    application.add_handler(CommandHandler("today", today))
    application.add_handler(CommandHandler("check", check))
    application.add_handler(CallbackQueryHandler(button_callback))
    application.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND,
    handle_message))

    # Запускаем бота
    print("Бот готов к работе! Напишите @ваш_username_бота в Telegram")
    application.run_polling()

if __name__ == '__main__':
    main()

```